Graph-Based Machine Learning Pipelines and Automatic Loop Parallelization in Accelera

Accelera Pipe and Parallelizer Modules

Mohamed Ashraf

ID: 9220658

Supervisor: Dr. Salma Abdel monem

June 2026

Abstract

This paper describes two core modules of Accelera: `accelera_pipe`, a graph-based machine learning pipeline executor, and the Parallelizer, a source-to-source loop optimization module that emits OpenMP-enabled C++ for supported code patterns. The pipeline module represents preprocessing, model fitting, prediction, metrics, branching, merging, and executed inference graphs as a native backend graph. This representation enables parallel branch execution, path selection, graph reuse, and targeted memory optimizations. The Parallelizer converts supported Python code to C++, extracts loops, classifies safe OpenMP transformations, inserts pragmas, and can compile custom preprocessing functions into Python-callable native extensions. Experiments show that `accelera_pipe` preserves sklearn-equivalent accuracy while reducing latency on a large four-candidate model-selection benchmark from about 803 seconds to 495 seconds. The Parallelizer also accelerates hard loop examples and reduces a preprocessing-heavy Python kernel from 6.83 seconds to 0.08 seconds in a repeated cached end-to-end pipeline run.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Modern machine learning workflows often require repeated evaluation of several preprocessing and model candidates. A direct implementation usually fits each candidate pipeline independently, repeats transformations, and retains intermediate arrays longer than necessary. In parallel, user-defined preprocessing functions may contain numeric loops written in Python, which can become a major bottleneck when the input data grows.

Accelera addresses these two issues through two complementary modules. The first module, `accelera_pipe`, expresses machine learning workflows as a graph. Nodes represent preprocessing steps, models, predictions, metrics, merges, and branch alternatives. The graph backend executes independent branches in parallel and returns an executed graph that can be reused for inference.

The second module, the Parallelizer, analyzes loop-heavy source code and adds OpenMP pragmas when the loop structure is considered safe. For supported Python functions, a restricted Python-to-C++ converter emits C++ code before loop extraction. The generated code can then be compiled into a Python extension and inserted into an `accelera_pipe` preprocessing node.

This paper focuses only on these two modules. The main contributions described here are: a graph-based pipeline execution model, memory and branching optimizations for pipeline search, a hybrid rule and classifier loop-parallelization strategy, a documented Python-to-C++ subset, and an integration path that uses the Parallelizer as an acceleration backend for custom preprocessing functions.

Chapter 2

Literature Review

Chapter 3

Methodology

3.1 Accelera Pipe Module

3.1.1 Functional Description

The Accelera Pipe module is the high-level workflow construction layer of the system. Its main function is to let the user describe a machine-learning pipeline in Python while delegating the actual graph construction, scheduling, branch execution, and runtime data management to the native C++ backend. Instead of forcing the user to manually manage intermediate outputs between preprocessing, modelling, prediction, and evaluation stages, the module exposes a fluent builder interface where every call adds a new computational node to an internal directed acyclic graph (DAG).

From the user's point of view, the module behaves like a pipeline object. A user can add preprocessing steps, models, prediction nodes, metrics, merge operations, and alternative branches. Internally, however, these operations are not stored as a simple linear list. Each operation is converted into a graph node with a specific type such as `PREPROCESS`, `MODEL`, `PREDICT`, `MERGE`, or `METRIC`. This representation is important because it allows the same pipeline API to support both sequential workflows and branched workflows where multiple candidate paths can be evaluated in parallel.

The module also separates training-time execution from inference-time execution. When a pipeline is called, the native graph executes all required nodes and returns two categories of output: the computed predictions or metric results, and an `ExecutedGraph` object. The `ExecutedGraph` contains only the selected fitted execution path and can be reused later for inference. This is a practical design decision because training normally requires validation data, metric objects, temporary transformed arrays, and candidate branches, while inference should retain only the fitted transformations and models required to process new input data.

Another functional role of Accelera Pipe is transparent optimization of custom preprocessing logic. When the user provides a custom preprocessing function or a custom transformer, the pipeline sends it through the Code Parallelizer before storing it in the graph. Therefore, the user can write a normal Python preprocessing function, while the system attempts to replace loop-heavy parts with a compiled Python-callable C++ implementation when the transformation is safe and supported. If the optimization cannot be applied, the original function is kept, so the external behavior of the pipeline remains unchanged.

3.1.2 Modular Decomposition

The Accelera Pipe module is decomposed into a Python API layer and a native C++ graph execution layer. The Python layer is responsible for usability, builder semantics, metric wrapping, serialization calls, and integration with the Code Parallelizer. The C++ layer is responsible for graph storage, node connections, execution order, parallel scheduling, selected-path extraction, and memory lifetime management.

Table 3.1: Main components of the Accelera Pipe module

Component	Responsibility
Pipeline Builder Interface	Provides the user-facing fluent API used to define preprocessing stages, models, prediction steps, metrics, merge operations, and branches. It translates each user call into a typed computational node instead of immediately executing the operation.
Pipeline State and Persistence Manager	Owns the native graph object, maps high-level node categories to graph node types, controls graph execution settings, and provides save/load behavior for reusable pipeline objects.
Executed Graph Runtime	Represents the fitted execution path selected after training. It allows the same trained transformations and models to be reused for inference without keeping unnecessary validation data or temporary training branches.
Metric and Display Wrappers	Adapt supervised metrics, unsupervised metrics, arrays, figures, dictionaries, and reports into graph-compatible outputs that can be stored, selected, displayed, or used in branch-selection strategies.
Native Graph Engine	Stores the pipeline as a directed acyclic graph, validates node connections, computes execution order, schedules sequential or parallel execution, manages branch selection, and controls intermediate-memory lifetime.
Node Abstraction Layer	Defines the common behavior of input, preprocessing, model, prediction, merge, and metric nodes, including source-node relationships, stored data, GPU usage flags, and selected-path markers.

At the Python API level, each builder method creates or configures one graph operation. A preprocessing node stores a function or transformer together with execution parameters such as `execute_fit` and `cache`. Before a preprocessing function is stored, the module attempts to parallelize it through `parallelizer.parallelize`. Model nodes store estimators, prediction nodes store test data and the prediction method name, metric nodes store metric wrappers, and merge nodes store the merge strategy.

Table 3.2: Mapping from builder calls to graph node semantics

Builder method	Node type	Main role
`preprocess(name, func)`	`PREPROCESS`	Transform input data or selected columns
`model(name, model)`	`MODEL`	Fit or apply a machine-learning estimator
`predict(name, test_data)`	`PREDICT`	Run a model output function such as `predict`
`metric(name, metric)`	`METRIC`	Evaluate predictions using a supervised or unsupervi
`merge(name, strategy)`	`MERGE`	Combine outputs from multiple prediction branches
`branch(name, ...)`	Multiple nodes	Create alternative candidate paths in the DAG

The native backend uses a graph abstraction rather than a linear pipeline. The `Graph` class stores all nodes, tracks whether the graph has already been compiled, supports cloning, and exposes methods for adding nodes, splitting branches, executing, clearing, saving, loading, and enabling parallel execution. The graph computes a topological execution order before running. This guarantees that each node is executed only after its source nodes have produced their outputs. If the graph contains branches, the execution result is followed by path selection using one of the supported strategies: maximum metric, minimum metric, custom strategy, or all paths.

Algorithm 3.1: High-level Accelera Pipe training flow

Input: training data X, optional labels y, graph G, selection strategy S
Output: predictions or metric results R, executed graph E

1: Add X and y to the input node of G
2: Compile G if its topological order is not already available
3: Execute graph nodes sequentially or in parallel depending on G settings
4: Collect outputs from final leaf nodes
5: If G contains branches, select the path requested by S
6: Clone the selected fitted path into a compact executed graph E
7: Remove validation—only data and temporary training data
8: Return R and E

Branching is handled by the native `split` operation. Each branch is converted into a chain of graph nodes starting from the current leaf. The backend uses the declared node type and the stored Python object to instantiate each branch node. A branch may contain a single node or a list of nodes, so the same mechanism supports simple model comparison as well as full alternative sub-pipelines.

A useful optimization in branch construction is duplicate first-node sharing. If several branches begin with the same first node, the graph creates that first node once and connects the later branch consumers to its output. This is safe for the first node because all such branches receive the same original input. The optimization is deliberately conservative: later repeated nodes are not merged automatically because their inputs may already differ due to earlier branch operations.

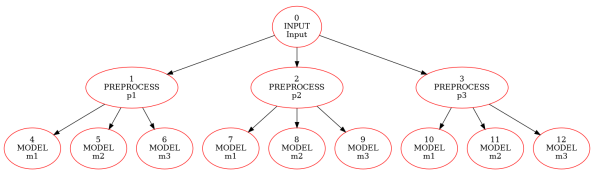


Figure 3.1: Before duplicate first-node sharing

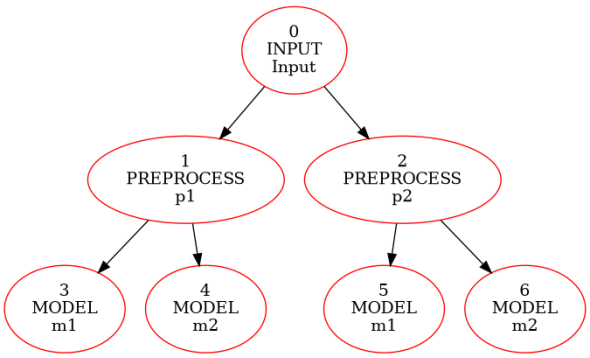


Figure 3.2: After duplicate first-node sharing

3.1.3 Design Constraints

The first design constraint is that the graph must remain a valid DAG. Pipeline execution depends on topological sorting, so cyclic dependencies cannot be allowed. Node connections are therefore restricted by node type. For example, a preprocessing node may follow the input node or another preprocessing node, a model node may follow input or preprocessing output, a prediction node must follow a model, a metric node must follow a prediction or merge node, and a merge node combines prediction outputs. These connection rules protect the execution engine from invalid graphs and preserve the semantic meaning of every stage.

The second constraint is safe memory management. Machine-learning pipelines can produce large intermediate arrays, especially when several preprocessing branches are evaluated. Accelera Pipe therefore tracks remaining consumers for each node output. Once the last downstream consumer finishes, the corresponding intermediate data can be released. Model and metric nodes are treated more carefully because they may hold fitted estimators or evaluation state, while input and preprocessing data can often be cleared after their consumers are done. This policy reduces memory pressure without changing the logical result of the graph.

Algorithm 3.2: Consumer-count based intermediate release

Input: executed node N, remaining consumer table C
Output: graph with unused temporary data released

1: for each source node S that feeds N do
2: $C[S] = C[S] - 1$
3: if $C[S] == 0$ and S is releasable then
4: clear the temporary data stored by S
5: end if
6: end for

The third constraint is safe handling of mutable preprocessing input. Some transformers may mutate the input array in place, while many scikit-learn transformers return a transformed array without modifying the original input. The backend therefore includes an inspection step that avoids unnecessary copies for ordinary scikit-learn objects but remains conservative for custom functions or objects that explicitly disable copying. This balances performance with correctness: unnecessary copying is avoided when possible, but unsafe aliasing is not assumed away.

The fourth constraint is parallel execution safety. The graph can run independent nodes in parallel, but a node can start only after all its parents have finished. The

backend uses CPU thread semaphores, a separate GPU semaphore, and dependency checks over the topological order. This prevents a downstream node from reading incomplete data and also serializes GPU work when necessary. The number of CPU threads is based on hardware concurrency but can be overridden using the `ACCELERA_NUM_THREADS` environment variable.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned, validation data stored inside prediction nodes is cleared, metric target values are removed, and temporary arrays in the training graph are released. This constraint is essential because the executed graph is intended to be saved or reused for inference, not to duplicate the whole training runtime state.

3.1.4 Other Description of Module

The Accelera Pipe module also supports persistence. The Python `PipelineBase` class saves and loads pipeline objects using pickle. The C++ graph exposes its state as a Python dictionary containing node information, selected-path flags, source indices, cached data, graph compilation state, and parallel execution configuration. During loading, the graph rebuilds the nodes and reconnects them using the stored source indices. This design makes the graph serializable even though part of the implementation is native C++.

The module can also export a graph-like representation of the pipeline structure. The graph utility code writes an XML-style network description containing layers for input, preprocessing, model, prediction, merge, and metric nodes, together with the edges between them. This representation is useful for debugging and for showing how the high-level pipeline has been lowered into an executable graph.

In summary, Accelera Pipe is not only a convenience wrapper over scikit-learn objects. It is a hybrid Python/C++ pipeline execution system. Python provides the user-facing construction interface, while C++ provides the graph semantics, execution scheduling, branch selection, and memory-lifetime control. This split is the main reason the module can remain easy to use while still supporting parallel execution and advanced graph-level optimizations.

3.2 Code Parallelizer Module

3.2.1 Functional Description

The Code Parallelizer module is responsible for transforming supported loop-heavy source code into optimized C++ code annotated with OpenMP pragmas. It is designed to reduce the execution time of custom preprocessing functions and standalone code snippets by identifying loops that can be safely executed in parallel. The module accepts different input forms: a file path, a source string, a custom Python function, or a custom transformer instance. Regardless of the input form, the scientific goal is the same: normalize the input into a C++ loop analysis path, detect parallelization opportunities, and return either parallelized source code or a compiled Python-callable native function.

For Python input, the module first lowers a restricted Python subset into C++. This conversion is intentionally limited to predictable language constructs such as arithmetic expressions, assignments, `for range(...)` loops, conditionals, function definitions, array-style indexing, and simple calls. The resulting C++ code is then analyzed by the same

loop extraction path used for native C/C++ input. This design avoids building two separate parallelization pipelines and ensures that both Python and C++ inputs are judged using the same loop features and safety checks.

Table 3.3: Supported operations in the Python-to-C++ converter

Category	Supported forms and notes
Constants and names	`None`, booleans, integers, floats, strings, and variables.
Arithmetic	Addition, subtraction, multiplication, division, modulo, and power using `std::pow`.
Unary and boolean	Unary plus, unary minus, logical not, logical and, and logical or.
Comparisons	Equality, inequality, less-than, less-than-or-equal, greater-than, and greater-than-or-equal; chained comparisons rejected.
Calls and printing	Ordinary function calls and `print`; keyword arguments are generally unsupported.
Access operations	Attribute access and indexing are supported; slicing is rejected.
Assignment	Single-target assignment and indexed assignment are supported; multi-target assignment is rejected.
Augmented assignment	Simple-name `+=`, `-=`, `*=`, `/=`, and `%=`.
Control flow	`if`/`else` blocks and `return` statements.
Loops	`for i in range(...)` loops with one, two, or three range arguments.
Functions	Function definitions are emitted as C++ template-style functions; decorators are rejected.

After normalization, the module extracts loop information using native Clang AST bindings. Each loop is represented by its source code and source-line range. The Python orchestration layer then computes a fixed loop-feature representation, uses a classifier endpoint to predict whether the loop is a parallelization candidate, and validates the decision with rule-based checks. If the loop is safe, the module injects an OpenMP directive such as `#pragma omp parallel for`. For reduction-like loops, it can also emit a reduction clause when the reduction variable is valid.

The module has a fail-safe behavior. If the input cannot be converted, analyzed, classified, compiled, or loaded as a native extension, the system falls back to the original Python function or leaves the code unmodified. This is important because parallelization is an optimization, not a reason to change the semantic meaning of the user's program.

3.2.2 Modular Decomposition

The Code Parallelizer module is decomposed into six cooperating components: input normalization, Python-to-C++ conversion, Clang-based loop extraction, feature extraction and vectorization, classification with rule-based validation, and OpenMP/native-extension generation.

Table 3.4: Main components of the Code Parallelizer module

Component	Responsibility
Parallelization Orchestrator	Receives source strings, file paths, custom functions, and transformer instances. It coordinates conversion, loop extraction, feature analysis, classification, pragma insertion, caching, compilation, and fallback behavior.
Input Normalization Layer	Converts accepted input forms into a common source-analysis representation. Files are read, source strings are analyzed directly, functions are captured from source, and transformer instances are inspected through their transformation method.
Python-to-C++ Converter	Translates a restricted subset of Python into C++ so Python preprocessing logic can enter the same static loop-analysis pipeline as native C/C++ code.
Loop Extraction Engine	Uses a compiler-style front end to locate loops, record source ranges, and expose structured loop information for feature extraction and rewriting.
Feature Extraction and Embedding Generator	Computes structural, dependency, memory-access, control-flow, arithmetic, and reduction features, then maps them into a fixed-size numeric representation.
Parallelization Classifier and Safety Resolver	Combines classifier predictions with deterministic safety rules to decide whether a loop remains sequential, receives a parallel-for pragma, or receives a reduction pragma.
OpenMP Rewriter	Inserts the selected OpenMP directive while avoiding unsafe duplicate insertion, especially when an outer loop already contains selected inner loops.
Native Compilation and Loading Layer	Wraps optimized C++ into a Python-callable extension, adds OpenMP flags only when needed, builds the native module, loads it at runtime, and returns the optimized callable.
Caching Layer	Stores processed source and compiled native functions using source-based hashes to avoid repeated conversion and compilation.

Figure 3.3 summarizes the overall data flow. In general, supported inputs are normalized into a common C++ loop-analysis path, while callable Python inputs additionally proceed to PyBind11 extension generation.

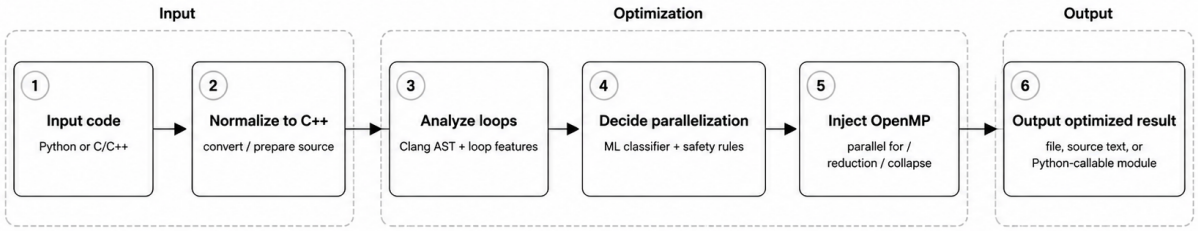


Figure 3.3: Code Parallelizer module workflow

The feature extraction stage analyzes both code structure and loop behavior, including reductions, array accesses, dependencies, indirect memory access, early exits, nesting depth, branches, function calls, arithmetic operations, and vectorization potential.

These indicators are encoded into a 40-dimensional vector for the classifier, then refined by deterministic safety rules that can still detect safe cases such as reductions, independent array writes, and nested loop patterns.

Algorithm 3.3: Loop classification and pragma insertion

```
Input: source program P
Output: source program P' with safe OpenMP pragmas

1: if P is Python source then
2:     convert the supported Python subset into C++
3: end if
4: Extract for−loops using the Clang AST frontend
5: for each extracted loop L do
6:     compute structural, memory, dependency, and reduction features
7:     send the 40−feature vector to the classifier service
8:     combine the predicted class with rule−based safety checks
9:     if the resolved class is parallel_for then
10:        insert #pragma omp parallel for
11:    else if the resolved class is reduction then
12:        insert #pragma omp parallel for reduction(...)
13:    else
14:        leave the loop unchanged
15:    end if
16: end for
17: Cache and return the transformed source code
```

The compilation path is used when the input is a Python callable. The parallelizer extracts or receives the function source, converts it to C++, parallelizes the C++ loop body, wraps the result in a PyBind11 module, compiles it as a shared native extension, loads the extension, and returns the optimized function. Caching is used at both the transformed-source level and the compiled method level. The cache key includes the compilation cache version, the function name, the compiler optimization level, the extension suffix, and the source code. This prevents recompiling the same function repeatedly.

3.2.3 Design Constraints

The most important design constraint of the Code Parallelizer is correctness. OpenMP pragmas can improve performance, but an incorrect pragma can produce wrong numerical results. Therefore, classifier output is not applied blindly. The module applies additional safety rules to avoid unsafe transformations when there are loop-carried dependencies, indirect accesses, early exits, side-effect function calls, or invalid reduction variables. Inner loops are also skipped when an outer selected loop already covers their range, preventing nested duplicate pragma insertion.

The second constraint is the restricted Python subset. Full Python semantics are too dynamic to translate safely into C++ using a lightweight source converter. The converter therefore rejects unsupported syntax such as decorators, slicing, chained comparisons, unsupported operators, non-range loop iterators, multi-target assignment, complex dynamic dispatch, and unsupported expression forms. This restriction is not a weakness in the design; it is a correctness boundary. The system optimizes code only when it can produce predictable C++.

The third constraint is compiler and platform compatibility. Native callable optimization depends on a working C++ compiler, Python include paths, PyBind11 headers, NumPy-compatible array wrapping, and platform-specific shared extension suffixes. The compiler path uses C++17 and adds `-fopenmp` only when the parallelized code actually contains OpenMP pragmas. On non-Windows platforms it also uses position-independent code flags suitable for shared libraries. If compilation fails, the original Python callable is returned.

The fourth constraint is input and output compatibility. The current compiled callable

path is oriented toward functions with one named input parameter and NumPy-like two-dimensional numeric data. The wrapper rewrites array access to PyBind11 unchecked NumPy access and exposes the compiled function back to Python. This makes the generated extension useful for numeric preprocessing loops, but it also means that arbitrary Python functions with complex object structures are outside the supported optimization target.

The fifth constraint is reproducibility and caching. Because native compilation can be expensive, the module caches processed C++ code and compiled functions using source hashes. The cache must include enough information to avoid stale or incorrect reuse when compiler options, cache version, function name, source code, or extension suffix changes. This design improves repeated execution time while keeping cache invalidation explicit.

3.2.4 Other Description of Module

The Code Parallelizer is directly integrated with Accelera Pipe. When a preprocessing node is added, the pipeline calls the parallelizer before storing the preprocessing object. If the object is a custom Python function, it is wrapped as a source-backed function whose runtime callable can be replaced by the compiled implementation. If the object is a custom transformer instance, the parallelizer attempts to optimize its `transform` method and attach the optimized method back to the instance. If the object is a library transformer or an unsupported callable, it is returned unchanged.

Figure 3.4: Integration flow for compiled preprocessing inside Accelera Pipe

Scientifically, the final design uses a hybrid approach rather than a purely generative code model. An earlier generator-based design attempted to emit parallelized code using a neural sequence model. That approach was rejected because the dataset size was not sufficient for reliable sequence generation and because generated synthetic samples risked introducing distribution bias. The current design is more controlled: the classifier only predicts the type of parallelization opportunity, while deterministic compiler-like rules decide how and whether the pragma is inserted. This separation makes the system easier to inspect, easier to debug, and safer for user code.

Overall, the Code Parallelizer acts as an optimization layer around user-defined numeric loops. It does not attempt to parallelize all Python programs. Instead, it targets the subset where static analysis, feature extraction, classifier prediction, rule validation, and native compilation can cooperate safely. This fits the broader Accelera design philosophy: keep the Python API simple, but move performance-sensitive and structurally analyzable work into optimized native execution when the system can do so without changing program semantics.

3.3 Accelera Pipe Design

`accelera_pipe` is implemented as a Python API over a native C++ graph backend. The Python layer builds the graph through high-level methods, while the backend stores node

Table 3.5: Main `accelera_pipe` builder operations.

Method	Role
`preprocess`	Adds a transformer or custom preprocessing function.
`model`	Adds a sklearn-compatible or custom trainable model.
`predict`	Stores the prediction operation and evaluation data.
`metric`	Computes a supervised or unsupervised metric.
`branch`	Inserts alternative graph paths from one split point.
`merge`	Combines branch predictions, currently through hard voting.

connectivity, branch structure, execution state, and fitted objects. The main user-facing class is `Pipeline`; after execution, a fitted graph is wrapped by `ExecutedGraph`.

The main pipeline operations are summarized in Table 3.5. A normal node is appended immediately to the graph. When `branch=True`, the method returns a lightweight branch node object, and `branch()` later inserts one or more alternatives into the graph.

During training, the pipeline receives `X` and optionally `y`. Preprocess nodes fit and transform data when needed, model nodes fit estimators, prediction nodes evaluate validation or test data, and metric nodes produce selection values. The graph may return all paths or select one path using `all`, `max`, `min`, or a custom selection strategy.

3.4 Executed Graphs and Persistence

After a training run, the backend clones the fitted selected graph into an executed graph. This executed graph keeps the fitted preprocessors and models required for inference, but the original training graph is cleaned from temporary runtime data and fitted runtime objects. This separation keeps the unexecuted pipeline reusable for future training while the executed graph remains an inference artifact.

Both unexecuted and executed graphs can be saved as one pickle file. Native graph pickling is exposed through pybind, where the C++ graph is converted to a Python dictionary and reconstructed during load. For custom Python preprocessing functions and lambdas, Accelera wraps the callable in a source-backed object when possible. The wrapper stores the function source code and reconstructs the callable with Python execution at load time. Closures are rejected because hidden captured variables cannot be safely reconstructed from source alone.

3.5 Pipeline Memory Optimizations

Branch search creates pressure on memory because several candidate paths may hold transformed arrays and fitted states at the same time. The current implementation includes four important optimizations.

First, caching is optional and disabled by default. This avoids the overhead of hashing, loading, and dumping intermediate node outputs during ordinary experiments. Second, the backend uses consumer-count based release. Each node output has a count of downstream consumers. After a consumer finishes, the count is decremented; when it reaches zero, the source node's temporary data can be cleared. Third, preprocessing input copying is guarded. The helper `shouldCopyPreprocessInput()` copies custom callables

conservatively, but skips defensive copies for sklearn transformers whose parameters indicate that they do not mutate input in place. Fourth, after creating the executed graph, the training graph is cleaned so raw training arrays and fitted runtime state do not remain attached to the reusable pipeline object.

3.6 Core Pipeline Algorithms

Algorithm 3.4 summarizes the execution flow. The important design point is that the training graph and the executed graph have different responsibilities. The training graph describes what should be run again, while the executed graph stores fitted objects needed for inference.

Algorithm 3.4: Pipeline training and executed-graph creation.

```
Input: training data X, optional labels y, selection strategy
Output: result records and executed graph

attach X and y to the graph input node
execute graph nodes in dependency order
for each preprocess node:
    fit/transform using its incoming data
for each model node:
    fit model using incoming X and y
for each predict node:
    apply fitted path to validation/test data
for each metric node:
    compute metric result
select path using all/max/min/custom strategy
clone fitted selected state into ExecutedGraph
clear runtime data and fitted objects from training graph
return results and ExecutedGraph
```

Algorithm 3.5 describes the consumer-count release optimization. It prevents arrays from staying alive after the last downstream node has consumed them.

Algorithm 3.5: Consumer-count based intermediate release.

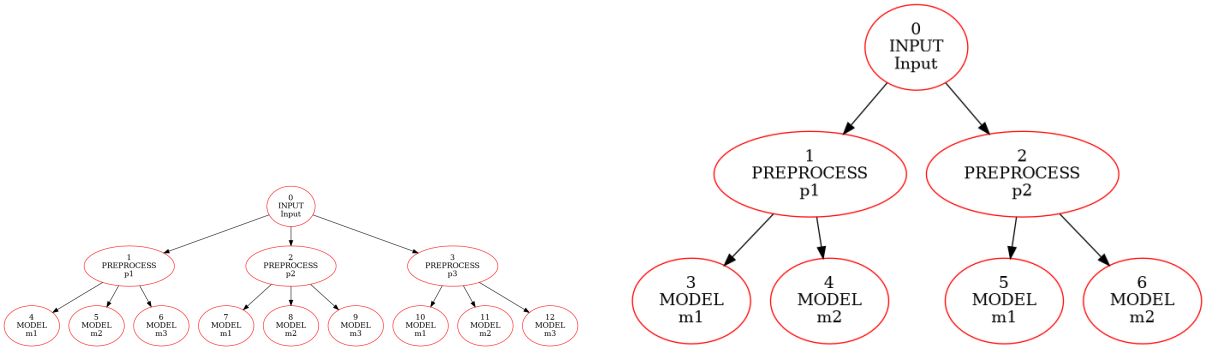
```
Before execution:
    consumer_count[node] = number of outgoing consumers

After node N executes:
    for each source S of N:
        consumer_count[S] −= 1
        if consumer_count[S] == 0 and S is releasable:
            clear temporary X/y/result data stored in S
```

3.7 Branch Sharing

The graph also avoids duplicated work when a split contains repeated equivalent first nodes. If a branch is declared as `MinMaxScaler`, `StandardScaler`, `MinMaxScaler`, the split creates only one MinMax node and one Standard node. The sharing signature depends on node type and object identity/representation rather than the visible node name, so duplicate operations can be shared even when named differently.

This sharing is intentionally restricted to the first node of each branch path. In list branches such as `[p1, p2, p3]` and `[p4, p2, p3]`, the two `p2` nodes must remain separate because their inputs are produced by different previous nodes. Figure 3.5 shows

the effect of sharing at the branch entry.



(a) Before duplicate first-node sharing

(b) After duplicate first-node sharing

Figure 3.5: Branch graph before and after sharing equivalent first branch nodes.

3.8 Parallelizer Design

The Parallelizer is implemented mainly in `accelera/src/utils/parallelizer.py`. It uses native Clang-backed bindings to extract loops from C/C++ code and uses a restricted Python-to-C++ converter for supported Python functions. For each loop, the module extracts static features describing reductions, dependencies, memory access, branches, function calls, nested depth, and estimated loop shape.

The build system treats LLVM/Clang as a required dependency for the Parallelizer frontend. To reduce manual setup, CMake first searches for an existing LLVM/Clang installation and then attempts automatic installation when possible. On Debian/Ubuntu Linux, it invokes the project installer through root or non-interactive sudo. On Windows, it attempts installation through `winget` and then Chocolatey. After installation, CMake re-runs package detection and builds the Clang-backed pybind module when the libraries are available. The runtime compiler used for optimized preprocessing functions also prefers LLVM `clang++` on Windows and avoids Linux-only flags such as `-fPIC`.

The loop decision system is hybrid. A classifier service receives a 40-dimensional loop feature vector and predicts one of three labels: `none`, `parallel_for`, or `reduction`. The classifier is stored under `models/open_mp_classifier`; the service loads `model.pkl` and `label_encoder.pkl`. The model is an `XGBClassifier`, and the label encoder maps numeric predictions back to the textual OpenMP classes.

The classifier is not treated as the only source of truth. Deterministic rules can recover common safe cases such as scalar reductions, independent array writes, and consecutive nested loops. The final generator emits `#pragma omp parallel for`, optional `collapse(n)`, and scalar `reduction` clauses.

Algorithm 3.6 shows the complete source-to-source path. The classifier proposes a class, but the resolver can override `none` when deterministic evidence proves a common safe pattern.

Algorithm 3.6: Parallelizer loop annotation algorithm.

```
Input: C/C++ file or supported Python code
Output: C++ code with OpenMP pragmas

if input is Python:
    convert supported Python AST to C++
extract loops using Clang−backed bindings
for each extracted for−loop:
    features = extract_static_features(loop)
    vector = vectorize_features(features)
    predicted = classifier.predict(vector)
    final_class = resolve_with_rules(loop, predicted)
    if final_class == parallel_for:
        insert "#pragma omp parallel for"
    if final_class == reduction:
        detect variable and operator
        insert OpenMP reduction pragma
format and return generated C++ code
```

3.9 Rule-Based Rationale

An earlier generator trial attempted to learn OpenMP pragma generation as a sequence-to-sequence task. The generator used a custom PyTorch encoder-decoder architecture with a bidirectional LSTM encoder, an attention-based LSTM decoder, teacher forcing, cross-entropy loss, Adam optimization, and gradient clipping. Its tokenizer was a custom BPE tokenizer configured with special tokens for padding, unknown tokens, sequence start, and sequence end.

This direction was not selected as the primary implementation for two reasons. First, the available corpus size was too small for robust code generation; the limited data distribution made the model prone to memorization and overfitting. Second, extending the dataset with AI-generated examples introduced distribution bias, because synthetic programs can repeat the patterns and style of the generating tool rather than representing real user code. A rule-based and feature-driven design was therefore preferred for the current system because it is more predictable, auditable, and safer for semantic-preserving OpenMP insertion.

3.10 Python-to-C++ Converter Support

The converter uses Python's `ast` module and intentionally supports only a small subset suitable for numeric loops. Unsupported syntax raises `ValueError`. Table 3.6 lists the supported operations.

Figure 3.6: Simplified flow for parallelizing a Python method or transformer and using it inside an Accelera Pipe graph node.

Table 3.6: Supported operations in the Python-to-C++ converter.

Category	Supported forms and notes
Constants and names	`None`, bool, int, float, string, variables.
Arithmetic	`+`, `-`, `*`, `/`, `%`, `**`; power uses `std::pow`.
Unary/boolean	unary `+`, unary `-`, `not`, `and`, `or`.
Comparisons	`==`, `!=`, `<`, `<=`, `>`, `>=`; chained comparisons rejected.
Calls/printing	Ordinary calls and `print`; keyword arguments are generally unsupported.
Access	Attribute access and indexing; slices rejected.
Assignment	Single-target assignment and indexed assignment; multi-target rejected.
AugAssign	Simple-name `+=`, `-=`, `*=`, `/=`, `%=`.
Control flow	`if`/`else` and `return`.
Loops	`for i in range(...)` with one, two, or three range arguments.
Functions	`def f(...)` as C++ template functions; decorators rejected.

Table 3.6 is intentionally narrow. The converter is not a general Python compiler; it is a practical bridge for loop-heavy preprocessing code. This restriction is important for correctness. By rejecting unsupported constructs early, the Parallelizer avoids silently generating C++ that changes Python semantics. The supported subset is enough for examples such as row normalization, scalar reductions, simple indexing, and range-based nested loops.

3.11 Integration Between the Two Modules

The two modules meet at custom preprocessing. If `Pipeline.preprocess()` receives a custom Python function, the pipeline attempts to optimize it with `parallelizer.optimize_pymethod`. If it receives a custom transformer object, the transformer instance can be optimized through `optimize_pyinstance` for supported preprocessing methods. The optimization is best-effort: failed conversion, unsupported syntax, compilation failure, or module loading failure falls back to the original Python callable.

Figure 3.6 summarizes the integration at a higher level. The preprocessing callable is converted to C++ when it belongs to the supported subset, the loop analysis stage adds OpenMP parallelism, and the compiled module is attached back to the graph node. The graph therefore keeps the same node structure while replacing eligible Python loops with native execution.

Chapter 4

Experiments and Results

4.1 Controlled Pipeline Comparison

The main `accelera_pipe` experiment compares three implementations of the same four-candidate model-selection task: a manual sklearn loop, sklearn `Pipeline`, and `accelera_pipe`. Each implementation evaluates the same preprocessing alternatives, `StandardScaler` and `MinMaxScaler`, and the same model alternatives, `LogisticRegression` and `SVC`. Candidate selection is performed on validation accuracy, and final accuracy is reported on the test split only once.

Tables 4.1 and 4.2 are the main scalability results. The benchmark starts with 1,000 total samples and grows to 250,000 total samples. Every row keeps the same protocol: 60% training, 20% validation, and 20% test data. Green highlights the fastest runtime for each sample size, while blue highlights the lowest memory usage.

Table 4.1: Latency scaling in seconds.

Samples	sklearn	sk Pipe	Accelera
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 4.1 shows the overhead boundary. At 1,000 samples, sklearn Pipeline finishes in 0.05 seconds, while `accelera_pipe` takes 0.34 seconds. This is expected because the native graph has setup, node dispatch, and branch-management overhead that cannot be amortized by such a small workload. Starting from 5,000 samples, the pattern changes: `accelera_pipe` becomes slightly faster than both sklearn baselines, and the advantage increases as the dataset grows. At 50,000 samples, it reduces runtime from 23.25 seconds for sklearn Pipeline to 14.81 seconds. At 250,000 samples, the reduction is much larger: from 803.01 seconds to 494.64 seconds.

Table 4.2 tells the opposite side of the design. sklearn Pipeline has the lowest RSS delta in every row because it runs one candidate pipeline at a time and releases many temporary objects naturally through Python object lifetime. `accelera_pipe` keeps graph nodes, branch outputs, and fitted branch state alive during execution, so its RSS delta is higher. This is why the memory optimizations in the methodology are important: consumer-count

Table 4.2: RSS memory delta scaling in MB.

Samples	sklearn	sk Pipe	Accelera
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

release, guarded copy removal, duplicate first-node sharing, and training-graph cleanup directly target the gap shown in Table 4.2.

Table 4.3: Largest controlled comparison run.

System	Val.	Test	Time	RSS
sklearn	0.9774	0.9768	803.56	303.95
sklearn Pipe	0.9774	0.9768	803.01	28.52
`accelera_pipe`	0.9774	0.9768	494.64	594.86

Table 4.3 isolates the largest run from the scaling experiment because it is the clearest evidence of the latency benefit. The dataset contained 150,000 training samples, 50,000 validation samples, and 50,000 test samples, each with 25 features. All three systems selected an equivalent `StandardScaler` followed by `SVC` path and achieved the same test accuracy, 0.9768. This means the runtime improvement is not caused by evaluating a different model, skipping validation, or changing the final test protocol. Under identical candidate selection behavior, `accelera_pipe` reduced runtime by 308.91 seconds relative to manual sklearn and 308.36 seconds relative to sklearn Pipeline. The table also makes the current limitation explicit: the faster graph execution pays a memory cost, reaching 594.86 MB RSS delta on this run.

4.2 OpenMP Classifier Evaluation

The Parallelizer classifier is evaluated as a three-class loop classifier. Its output is combined with local rules during pragma generation, but the classifier metrics still describe the quality of the learned loop-feature representation.

The classifier is not expected to be perfect because the final system still applies deterministic checks after prediction. However, Table 4.4 is useful because it shows whether the learned feature vector can separate the three main loop categories. The `parallel_for` class has the highest precision at 0.86, which is important because false positive parallelization can be unsafe. The `reduction` class has lower support, only 775 examples, but still reaches 0.82 recall. This suggests that the feature extractor captures common scalar-reduction patterns well enough for the classifier to help, while the rule layer remains necessary for safety.

The macro and weighted averages are both close to 0.83 F1-score. The macro average treats the three classes equally, so it is sensitive to the smaller `reduction` class. The weighted average follows the dataset distribution, where `none` and `parallel_for` are more common. The closeness of these two averages indicates that the model is not only performing on the majority classes. In the full Parallelizer, these predictions are used as a first signal, then local rules refine the decision before any OpenMP pragma is emitted.

Table 4.4: OpenMP classifier test metrics.

Class	Prec.	Rec.	F1	Support
`none`	0.82	0.85	0.84	3048
`parallel_for`	0.86	0.81	0.83	2696
`reduction`	0.79	0.82	0.80	775
Accuracy			0.83	6519
Macro avg.	0.82	0.83	0.83	6519
Weighted avg.	0.83	0.83	0.83	6519

4.3 Parallelizer Hard-File Evaluation

The hard-file evaluation script parallelizes examples from `data/hard_test_files`, compiles both baseline and generated versions, runs each executable three times, compares outputs, and records timing. Numeric outputs use tolerant comparison because OpenMP reductions can change floating-point accumulation order. Table 4.5 shows the measured results.

Table 4.5: Hard-file Parallelizer evaluation.

File	Base (ms)	Par. (ms)	Speedup
`hard_eval_000.cpp`	1786.28	195.47	9.138×
`hard_eval_001.cpp`	63.98	31.47	2.033×
`hard_eval_002.py`	42.66	7.67	5.563×

Table 4.5 evaluates the generated code as executable programs, not only as text. This matters because a pragma can look correct syntactically while still changing output values or failing to compile. The first file obtains the largest speedup, 9.138×, because its workload benefits strongly from parallel loop execution. The second file still improves, but only by 2.033×, which suggests either a smaller loop body, more overhead relative to useful work, or less exploitable parallel work. The third file begins as supported Python, passes through the Python-to-C++ converter, and still reaches 5.563×, which validates the Python conversion path in addition to the C++ pragma insertion path.

The three files were parallelized, compiled, executed, and verified successfully. The report contained 11 extracted loops, 8 gold pragmas, and 8 generated pragmas. Exact pragma matching reached 8 out of 8, and average pragma similarity was 1.0. This means that for these hard examples, the generated pragma text matched the expected OpenMP annotations. The average measured parallelization latency was 231.53 ms, which is small compared with the runtime savings for long-running kernels. Across the three verified files, the average speedup was 5.578×.

4.4 Pipeline Integration Benchmark

The integration experiment uses a custom row-normalization preprocessing function over an input shape of `(500000, 25)`. The Python implementation contains nested loops that compute the row norm and divide each feature by the norm. The optimized path converts the function to C++, inserts OpenMP, compiles a pybind module, and uses the compiled callable inside `accelera_pipe`.

Table 4.6: Automatic custom preprocessing acceleration inside `accelera_pipe`.

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

The integration benchmark in Table 4.6 reports the direct `examples/parallel_accpipe.py` run after normalizing example execution to the repository root. The pure Python preprocessing function takes 6.83 seconds on 500,000 samples. The first optimized process in the measured sequence takes 5.88 seconds because process-local setup and cache lookup still appear in the end-to-end path. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The example validation confirmed that the optimized output matched the Python output. This result shows that the integration benefits from two cache layers: one that avoids repeating Python-to-C++ conversion and OpenMP insertion, and one that avoids recompiling the pybind extension.

4.5 Direct Example Verification Across Operating Systems

The final verification step runs the same demonstration scripts used by the project Makefile. These examples are important because they exercise the modules as a user would run them: graph pipeline execution, automatic preprocessing optimization, standalone source parallelization, and save/load persistence. The Linux and Windows measurements should not be compared as hardware-normalized benchmarks; they are environment verification runs. Their purpose is to confirm that the examples complete, preserve outputs or accuracy, and expose where compilation and operating-system overhead appear.

Table 4.7: Linux direct example-script verification runs.

Example	Baseline time (s)	Optimized time (s)	Validation
`accpipe_demo.py`	2.48	1.79	same test accuracy, 0.9260
`parallel_accpipe.py`, run 1	6.83	5.88	outputs match
`parallel_accpipe.py`, run 2	6.83	0.08	outputs match
`code_parallelizer_demo.py`, run 1	5.81	0.014	outputs match
`code_parallelizer_demo.py`, run 2	6.05	0.015	outputs match
`save_load.py`, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
`save_load.py`, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 4.7 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make so that the timing for each demonstration could be read separately. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`. The C path also reported about 0.26–0.27 seconds of compilation time outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 4.8: Windows direct example correctness and path timing.

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
`accpipe_demo.py`	normal	1.75 / 1.78	1.03	same test accuracy, 0.926
`parallel_accpipe.py`	isolated cache, run 1	6.66	6.00	outputs match
`parallel_accpipe.py`	isolated cache, run 2	6.58	0.09	outputs match
`code_parallelizer_demo.py`	isolated cache, run 1	5.59	1.04	outputs match
`code_parallelizer_demo.py`	isolated cache, run 2	5.61	1.05	outputs match
`save_load.py`	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 4.9: Windows compilation and process overhead.

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
`accpipe_demo.py`	normal	–	6.53	Graph example startup
`parallel_accpipe.py`	isolated cache, run 1	included	14.37	Cold compile path
`parallel_accpipe.py`	isolated cache, run 2	cached	8.33	Warm cache path
`code_parallelizer_demo.py`	isolated cache, run 1	0.87	8.74	Temporary executable
`code_parallelizer_demo.py`	isolated cache, run 2	1.04	9.00	Temporary executable
`save_load.py`	normal	–	2.71	Persistence example

Tables 4.8 and 4.9 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table can remain readable. The examples were run one by one in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold-cache behavior and the second run represents warm-cache behavior. The Windows build path also exercises the platform changes made for the Parallelizer: CMake searches for LLVM/Clang, attempts automatic installation through Windows package managers when needed, and the runtime compiler uses the LLVM `clang++` path without Linux-only flags.

The strongest cache effect appears in `parallel_accpipe.py`. The cold end-to-end Accelera path takes 6.00 seconds because it must extract the Python function source, convert it to C++, select safe loops, generate OpenMP pragmas, compile a pybind extension, link it, import it, and attach the compiled callable to the preprocessing node. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages. This verifies that Windows execution benefits from the same cache design as Linux, even though the process-level wall time remains higher.

The Windows `code_parallelizer_demo.py` runs mainly measure end-to-end toolchain latency rather than pure loop speed. Each run rebuilds a temporary executable, so the reported wall time includes compilation, process startup, OpenMP initialization, dynamic loading, and executable creation. The generated program itself reports only about 0.024–0.025 seconds for the timed main-body work, while Linux has lower launch and linking overhead, making its runtime closer to the measured kernel work.

Chapter 5

Conclusion

The two examined modules address complementary bottlenecks. `accelera_pipe` improves workflow-level execution by representing candidate pipelines as a graph, running branches in parallel, selecting paths through metrics, reusing executed graphs for inference, and reducing temporary data retention. The Parallelizer improves loop-level execution by converting supported Python code to C++, extracting loops, classifying OpenMP opportunities, and compiling supported custom preprocessing functions into native Python-callable modules.

The current results show that the graph executor can match sklearn accuracy while reducing large benchmark latency, and that the Parallelizer can produce substantial speedups for both standalone hard loop files and integrated preprocessing kernels. Future work should expand the literature review, broaden the Python-to-C++ subset, improve memory usage during branch-heavy searches, and refine compile-cache policy for interactive workflows.